

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Data Model - Task Entity 6
 - 2.2 Requirements Traceability Matrix 7
 - 2.3 Task Management Workflow 7
 - 2.4 CLI Interaction Sequence 8
 - 3. Detailed Technical Specifications & Business Rules 9
 - 3.1 Requirements Traceability 9
 - 3.2 Security Rules 11
 - 3.3 Data Models 11
 - 4. Project Governance & Structural Gaps 11
 - 4.1 Structural Gaps 11
 - 4.2 Remediation & Workflow 12
 - 5. Technical & Domain Glossary (Terminology Reference) 12

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a local productivity tool designed for task capture and lifecycle management via a command-line interface. It utilizes a flat-file JSON storage pattern located at `~/ .todos .json` to maintain task state, including unique sequential IDs and completion statuses, targeting high-performance local execution and machine-readable interoperability.

1.2 Maturity Assessment

The project is in a **REFINEMENT** state. While the functional core is well-defined with a 100% completeness score in extracted nodes, there are significant unresolved uncertainties regarding edge-case handling (invalid IDs, corrupted JSON, empty descriptions, and concurrency). Additionally, the absence of a 'Scope & Out-of-Scope' section creates a medium-severity structural gap that risks scope creep.

1.3 Technical Stack

- **Languages and Frameworks:** Python 3.8+
- **Storage:** Local JSON file (`~/ .todos .json`)

1.4 Architectural Constraints

- **Execution Latency:** All core operations (add, list, complete, remove) must execute in < 2 seconds.
- **Scalability:** Stable performance for 1000+ tasks in the storage file.
- **Storage Path:** Strict data persistence at `~/ .todos .json`.
- **Data Integrity:** Unique sequential integer IDs for each Task entity.
- **Output Format:** Human-readable default with a strict valid JSON alternative via `--json` flag.
- **User Success Rate:** 95% success rate for basic operations on first attempt.
- **Error Handling:** 80% reduction in user confusion through actionable error messages.

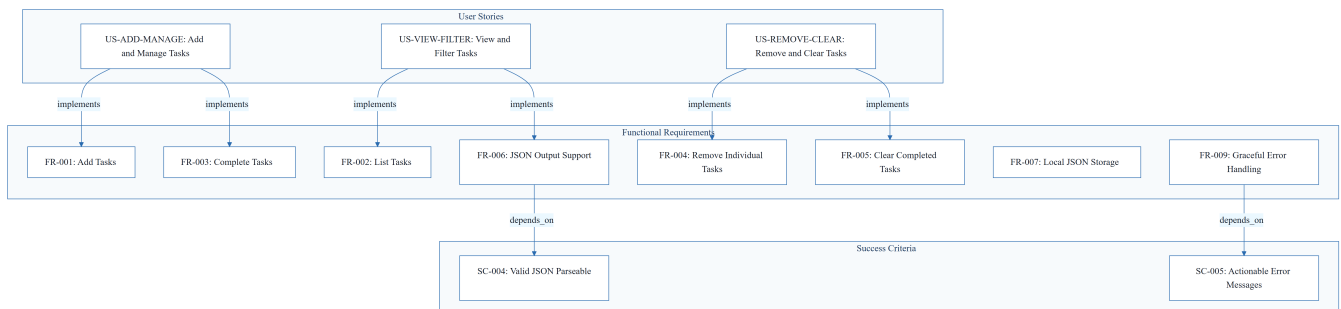
1.5 Critical Dependencies

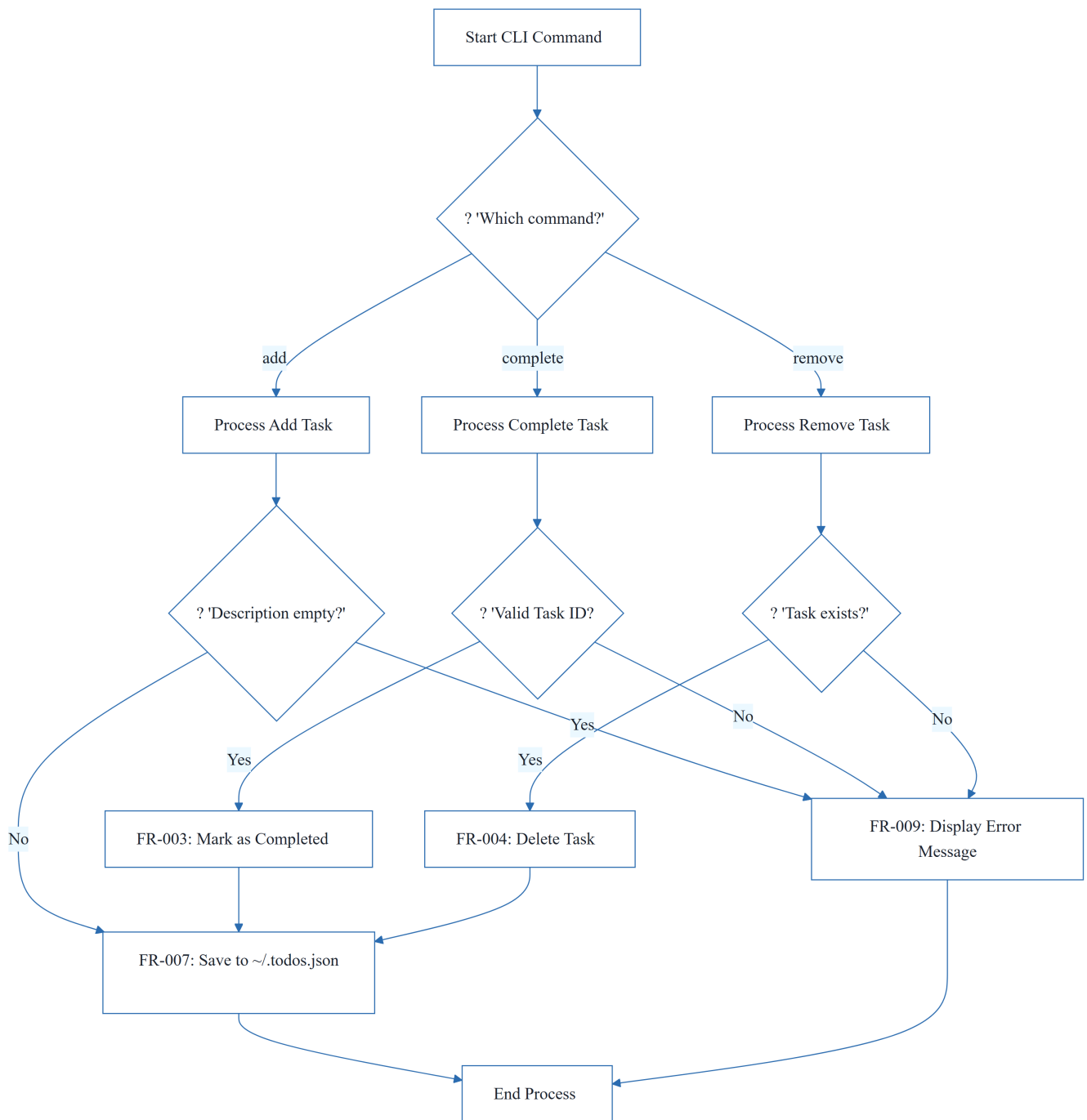
- Local filesystem write access to the home directory for `~/ .todos .json`.
- Python 3.8+ runtime environment.
- Task entity relational integrity: sequential ID generation linked to task creation.

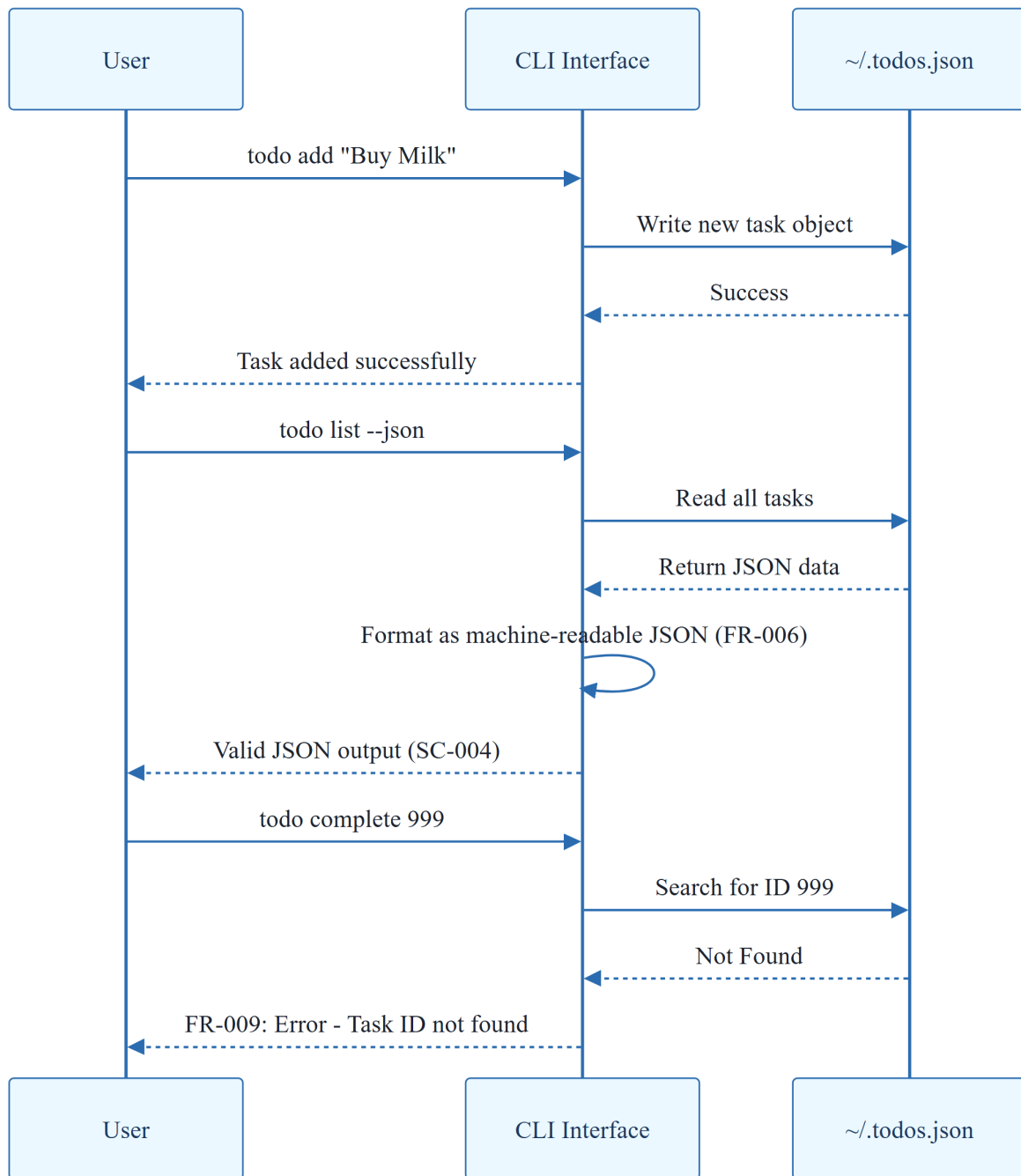
- JSON parser compatibility for both storage persistence and --json output.

2. Architecture Workflows

ENT-TAS]		
int	id	PK
string	description	
boolean	completed	
timestamp	created_at	





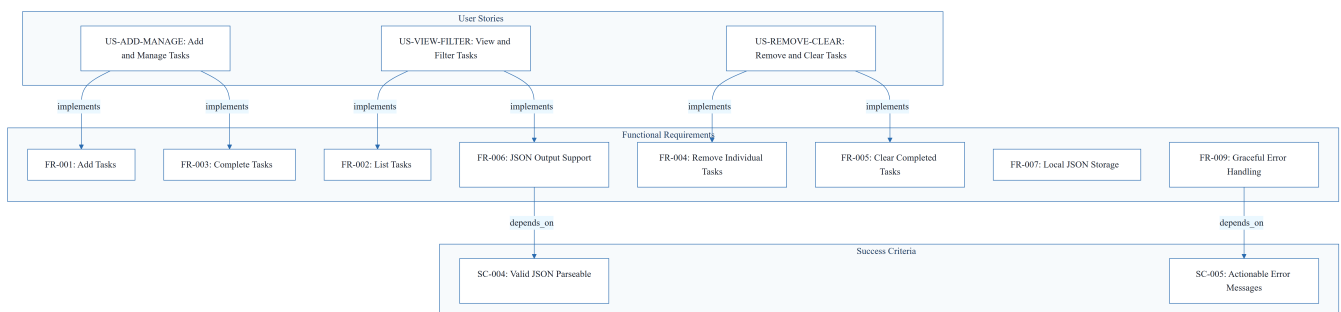


& Visual Diagrams

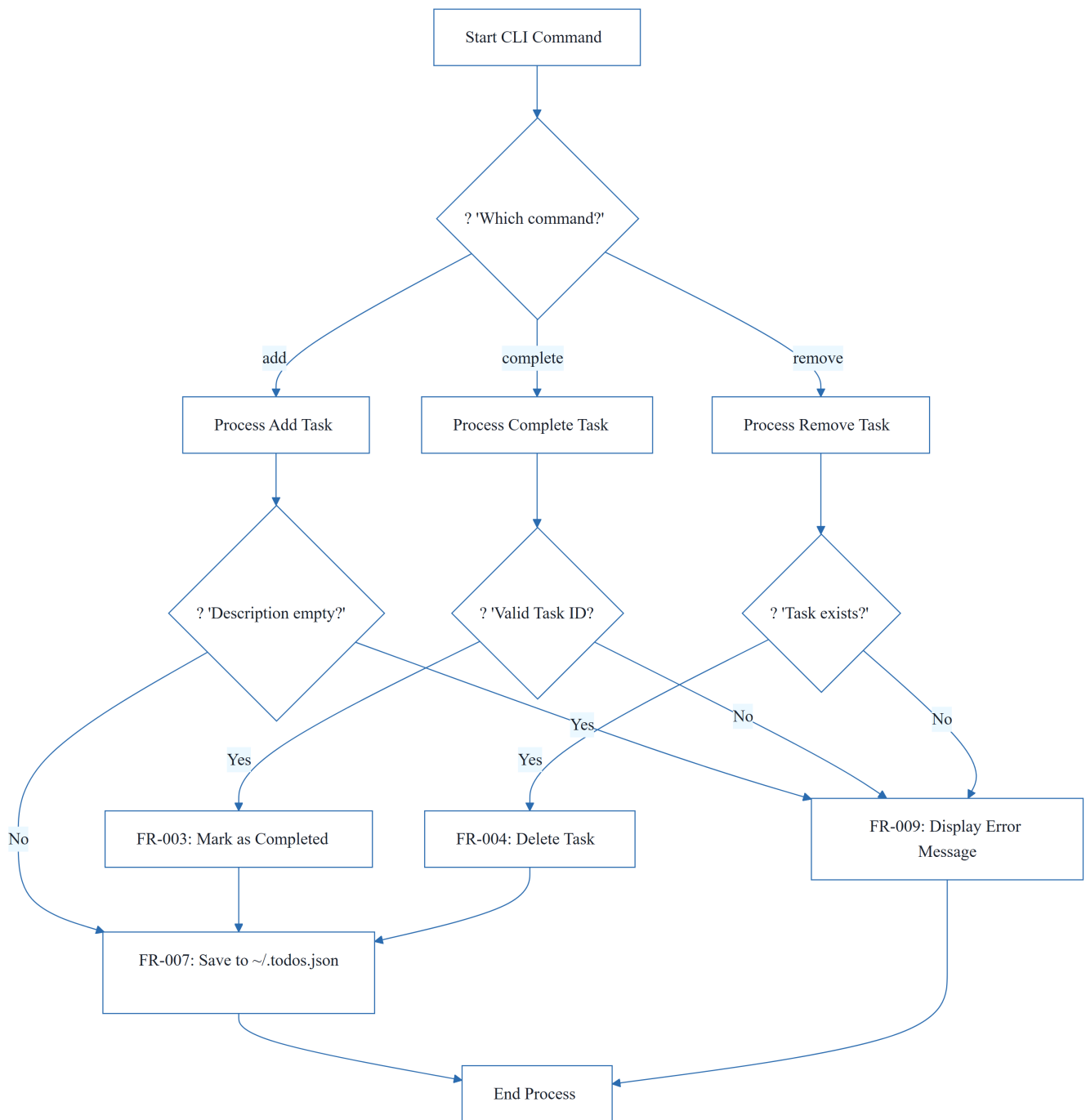
2.1 Data Model - Task Entity

ENT-TAS]		
int	id	PK
string	description	
boolean	completed	
timestamp	created_at	

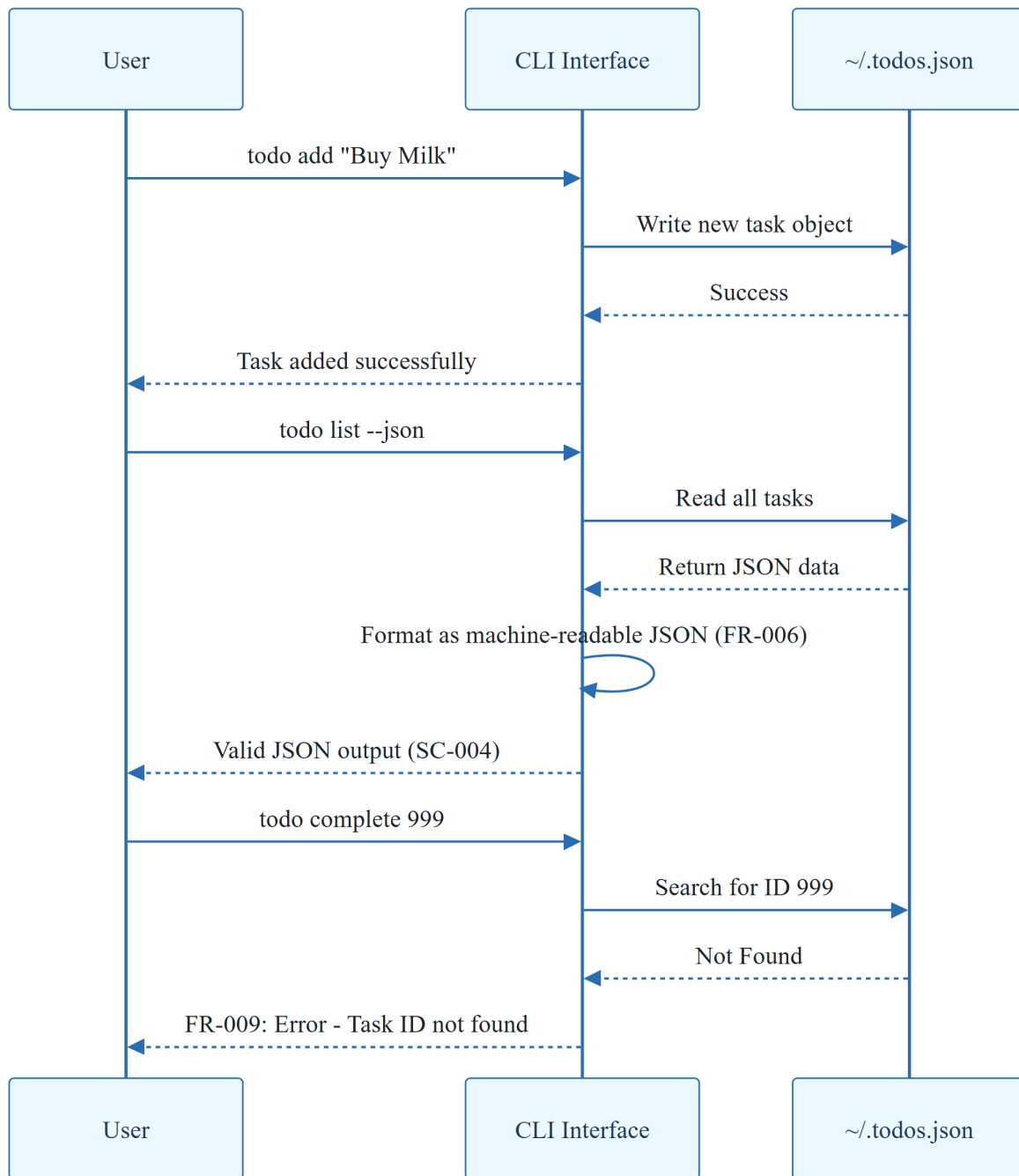
2.2 Requirements Traceability Matrix



2.3 Task Management Workflow



2.4 CLI Interaction Sequence



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Type	Description	Source User Story	Success Criterion
FR-001	Functional	System MUST allow users to add new tasks with descriptions via the add command	US-ADD-MANAGE	N/A

ID	Type	Description	Source User Story	Success Criterion
FR-002	Functional	System MUST display all tasks with their IDs, descriptions, and completion status via the <code>list</code> command	US-VIEW-FILTER	N/A
FR-003	Functional	System MUST allow users to mark tasks as completed via the <code>complete</code> command	US-ADD-MANAGE	N/A
FR-004	Functional	System MUST allow users to remove individual tasks via the <code>remove</code> command	US-REMOVE-CLEAR	N/A
FR-005	Functional	System MUST allow users to clear all completed tasks via the <code>clear</code> command	US-REMOVE-CLEAR	N/A
FR-006	Functional	System MUST support JSON output format via <code>--json</code> flag for machine-readable output	US-VIEW-FILTER	SC-004
FR-007	Functional	System MUST store all task data in a local JSON file at <code>~/ . todos . json</code>	N/A	N/A
FR-008	Functional	System MUST assign unique sequential IDs to tasks	N/A	N/A
FR-009	Functional	System MUST handle errors gracefully with user-friendly error messages	N/A	SC-005
SC-001	Success	Users can add, list, complete, and remove tasks in under 2 seconds on standard hardware	N/A	N/A
SC-002	Success	System handles 1000+ tasks in the JSON file without performance degradation	N/A	N/A

ID	Type	Description	Source User Story	Success Criterion
SC-003	Success	95% of users can successfully perform basic operations on first attempt	N/A	N/A
SC-004	Success	JSON output is valid and parseable by standard JSON parsers	N/A	N/A
SC-005	Success	Error messages are clear and actionable, reducing user confusion by 80%	N/A	N/A

3.2 Security Rules

- **Filesystem Access:** The application requires read/write permissions specifically for the home directory to manage `~/ .todos.json`.
- **Input Validation:** All CLI inputs must be sanitized to prevent malformed JSON injection or shell-related vulnerabilities.

3.3 Data Models

Entity: Task (ENT-TASK)

Field	Type	Description	Constraints
id	Integer	Unique identifier	Primary Key, Auto-incremented
description	String	Task content	Mandatory
completed	Boolean	Completion status	Default: <code>false</code>
created_at	Timestamp	Creation date/time	System generated

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap ID	Missing Section	Priority	Remediation Advice
GAP-01	Scope & Out-of-Scope	MEDIUM	Explicitly define what the CLI tool will NOT do (e.g., no cloud sync, no categories) to avoid scope creep.

4.2 Remediation & Workflow

To resolve the identified gaps and uncertainties, the following workflow is proposed:

1. **Scope Definition:** Define the boundary of the MVP to prevent feature creep.
2. **Edge Case Specification:** Define the exact behavior for:
 - Invalid IDs during complete or remove operations.
 - Recovery or error reporting for corrupted `~/todos.json`.
 - Validation rules for empty task descriptions.
 - Locking mechanism or error handling for concurrent file access.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
ID	BUSINESS_DOMAIN	FR-008	A unique sequential integer assigned to each registered item to ensure deterministic referencing.
JSON	TECHNICAL_STACK	FR-007	The lightweight data-interchange format used for persistent storage in the home directory and machine-readable output.
Python 3.8	TECHNICAL_STACK	ASSUM-PY38	The minimum required runtime environment version for executing the application logic.
todo clear	BUSINESS_DOMAIN	FR-005	An operation that purges all entries marked as finished while preserving those still pending.
todo list	BUSINESS_DOMAIN	FR-002	A command to retrieve and display the current set of registered items and their associated statuses.